

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Definite Integral Calculator & Visualizer</title>

  <script src="https://cdnjs.cloudflare.com/ajax/libs/mathjs/11.8.0/math.js"></script>

  <style>

    :root {

      --bg-color: #f8f9fa;

      --card-bg: #ffffff;

      --primary: #4a6fa5;

      --accent: rgba(74, 111, 165, 0.25);

      --text: #333333;

      --border: #dcdfe3;

    }

    body {

      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;

      background-color: var(--bg-color);

      color: var(--text);

      margin: 0;

      padding: 20px;

      display: flex;

      justify-content: center;

      align-items: center;

      min-height: 100vh;

      box-sizing: border-box;

    }
```

```
.container {  
  display: grid;  
  grid-template-columns: 360px 1fr;  
  gap: 20px;  
  width: 100%;  
  max-width: 1200px;  
  background: var(--card-bg);  
  padding: 20px;  
  border-radius: 12px;  
  box-shadow: 0 4px 20px rgba(0,0,0,0.08);  
}
```

```
@media (max-width: 900px) {  
  .container {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.controls {  
  display: flex;  
  flex-direction: column;  
  gap: 20px;  
  padding-right: 10px;  
}
```

```
h2 {  
  margin: 0 0 10px 0;  
  color: #2c3e50;  
  font-size: 1.5rem;  
}
```

```
.input-group {  
  display: flex;  
  flex-direction: column;  
  gap: 6px;  
}
```

```
label {  
  font-weight: 600;  
  font-size: 0.9rem;  
  color: #5a6c7d;  
}
```

```
input[type="text"], input[type="number"] {  
  padding: 10px;  
  border: 1px solid var(--border);  
  border-radius: 6px;  
  font-size: 1rem;  
  outline: none;  
  transition: border-color 0.2s;  
}
```

```
input:focus {  
  border-color: var(--primary);  
}
```

```
.range-inputs {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 10px;  
}
```

```
.slider-group {  
  display: flex;  
  flex-direction: column;  
  gap: 4px;  
  margin-top: 5px;  
}
```

```
input[type="range"] {  
  width: 100%;  
  margin: 10px 0;  
}
```

```
.result-box {  
  background-color: #edf2f7;  
  border-left: 4px solid var(--primary);  
  padding: 15px;  
  border-radius: 4px;  
  margin-top: 10px;  
}
```

```
.result-box .math-display {  
  font-size: 1.2rem;  
  font-family: "Cambria", "Times New Roman", serif;  
  font-style: italic;  
  margin-bottom: 5px;  
}
```

```
.result-box .value {  
  font-size: 1.6rem;  
  font-weight: bold;  
  color: var(--primary);  
}
```

```
}
```

```
.error {  
  color: #e53e3e;  
  font-size: 0.85rem;  
  min-height: 18px;  
}
```

```
.canvas-container {  
  position: relative;  
  width: 100%;  
  height: 500px;  
  border: 1px solid var(--border);  
  border-radius: 8px;  
  overflow: hidden;  
  background: #ffffff;  
}
```

```
canvas {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  cursor: grab;  
}
```

```
canvas:active {  
  cursor: grabbing;  
}
```

```

.instructions {
    font-size: 0.8rem;
    color: #718096;
    line-height: 1.4;
}
</style>
</head>
<body>

<div class="container">
    <div class="controls">
        <div>
            <h2>Definite Integral</h2>

            <p class="instructions">Type a math expression using standard notation (e.g., <code>x^2 - 1</code>, <code>sin(x)</code>, <code>log(x)</code>). Drag the canvas to pan around.</p>
        </div>

        <div class="input-group">
            <label for="fx">f(x) =</label>

            <input type="text" id="fx" value="x^2 - 1" placeholder="e.g., x^2 - 1">

            <div id="error-msg" class="error"></div>
        </div>

        <div class="range-inputs">
            <div class="input-group">
                <label for="limit-a">Lower limit (a)</label>

                <input type="number" id="limit-a" value="0" step="0.1">

                <div class="slider-group">
                    <input type="range" id="slider-a" min="-10" max="10" step="0.1" value="0">
                </div>
            </div>
        </div>
    </div>

```

```

<div class="input-group">
  <label for="limit-b">Upper limit (b)</label>
  <input type="number" id="limit-b" value="2" step="0.1">
  <div class="slider-group">
    <input type="range" id="slider-b" min="-10" max="10" step="0.1" value="2">
  </div>
</div>
</div>

```

```

<div class="result-box">
  <div class="math-display">∫<sub>a</sub><sup>b</sup> f(x) dx =</div>
  <div id="result-value" class="value">0.6667</div>
</div>
</div>

```

```

<div class="canvas-container">
  <canvas id="graphCanvas"></canvas>
</div>
</div>

```

```

<script>
  // Elements
  const fxInput = document.getElementById('fx');
  const inputA = document.getElementById('limit-a');
  const inputB = document.getElementById('limit-b');
  const sliderA = document.getElementById('slider-a');
  const sliderB = document.getElementById('slider-b');
  const resultValue = document.getElementById('result-value');
  const errorMsg = document.getElementById('error-msg');
  const canvas = document.getElementById('graphCanvas');
  const ctx = canvas.getContext('2d');

```

```
// Graph viewport state (Coordinates & Zoom scale)

let config = {
  zoomX: 40, // pixels per unit
  zoomY: 40,
  offsetX: 0, // dynamic offsets handled by centering and panning
  offsetY: 0
};
```

```
// User drag pan configurations

let isDragging = false;
let startX, startY;
```

```
// Set initial canvas sizing on load

function resizeCanvas() {
  const rect = canvas.parentElement.getBoundingClientRect();
  canvas.width = rect.width;
  canvas.height = rect.height;
  if(config.offsetX === 0 && config.offsetY === 0) {
    config.offsetX = canvas.width / 2;
    config.offsetY = canvas.height / 2;
  }
  update();
}
```

```
// Dynamic Simpson's Rule Integration Integration

function integrate(f, a, b, n = 1000) {
  if (a === b) return 0;
  // Adjust interval steps if boundaries are reversed
  const h = (b - a) / n;
  let sum = f(a) + f(b);
```

```

for (let i = 1; i < n; i++) {
    const x = a + i * h;
    sum += i % 2 === 0 ? 2 * f(x) : 4 * f(x);
}
return (h / 3) * sum;
}

```

// Primary Evaluation and Render function

```

function update() {
    errorMsg.innerText = "";
    const exprString = fxInput.value;
    let compiledExpr;

    // Try compiling input string using math.js
    try {
        compiledExpr = math.compile(exprString);
        // Test execution to handle bad syntax variables
        compiledExpr.evaluate({x: 1});
    } catch (e) {
        errorMsg.innerText = "Invalid mathematical function expression.";
        return;
    }
}

```

```

const f = (x) => {
    try {
        let res = compiledExpr.evaluate({x: x});
        return typeof res === 'number' ? res : NaN;
    } catch(e) {
        return NaN;
    }
}

```

```

};

const a = parseFloat(inputA.value) || 0;
const b = parseFloat(inputB.value) || 0;

// Calculate and update integral result value
const integralVal = integrate(f, a, b);
resultValue.innerText = isNaN(integralVal) ? "Undefined" : integralVal.toFixed(5);

drawGraph(f, a, b);
}

// Coordinate Conversion tools
function toScreenX(mathX) { return config.offsetX + mathX * config.zoomX; }
function toScreenY(mathY) { return config.offsetY - mathY * config.zoomY; }
function toMathX(screenX) { return (screenX - config.offsetX) / config.zoomX; }
function toMathY(screenY) { return (config.offsetY - screenY) / config.zoomY; }

// Canvas Plotting Layer Engine
function drawGraph(f, a, b) {
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  const w = canvas.width;
  const h = canvas.height;

  // Draw Grid Lines & Numeric labels
  ctx.strokeStyle = '#e2e8f0';
  ctx.lineWidth = 1;
  ctx.fillStyle = '#a0aec0';
  ctx.font = '10px Arial';

```

```

// Vertical grid lines (X axis)

const startXIndex = Math.floor(toMathX(0));

const endXIndex = Math.ceil(toMathX(w));

const stepX = (endXIndex - startXIndex) > 30 ? 5 : 1; // dynamically adapt densities

for(let x = Math.floor(startXIndex/stepX)*stepX; x <= endXIndex; x += stepX) {

    let screenX = toScreenX(x);

    ctx.beginPath();

    ctx.moveTo(screenX, 0);

    ctx.lineTo(screenX, h);

    ctx.stroke();

    if(x !== 0) ctx.fillText(x, screenX + 2, config.offsetY - 2);

}

// Horizontal grid lines (Y axis)

const startYIndex = Math.floor(toMathY(h));

const endYIndex = Math.ceil(toMathY(0));

const stepY = (endYIndex - startYIndex) > 30 ? 5 : 1;

for(let y = Math.floor(startYIndex/stepY)*stepY; y <= endYIndex; y += stepY) {

    let screenY = toScreenY(y);

    ctx.beginPath();

    ctx.moveTo(0, screenY);

    ctx.lineTo(w, screenY);

    ctx.stroke();

    if(y !== 0) ctx.fillText(y, config.offsetX + 2, screenY - 2);

}

// Draw Main Axes (X and Y basis lines)

ctx.strokeStyle = '#718096';

ctx.lineWidth = 2;

```

```

ctx.beginPath();

// X Axis
ctx.moveTo(0, config.offsetY); ctx.lineTo(w, config.offsetY);

// Y Axis
ctx.moveTo(config.offsetX, 0); ctx.lineTo(config.offsetX, h);

ctx.stroke();


// 1. Plot Shaded Area Under Curve between a and b
const minLimit = Math.min(a, b);
const maxLimit = Math.max(a, b);


ctx.fillStyle = 'rgba(74, 111, 165, 0.3)';
ctx.beginPath();

let initialPointSet = false;

// Step through individual pixel horizontal width to generate shaded path polygons
const leftScreenX = Math.max(0, toScreenX(minLimit));
const rightScreenX = Math.min(w, toScreenX(maxLimit));

for (let sx = leftScreenX; sx <= rightScreenX; sx++) {
  let mx = toMathX(sx);
  let my = f(mx);
  if (!isNaN(my) && isFinite(my)) {
    let sy = toScreenY(my);
    if (!initialPointSet) {
      ctx.moveTo(sx, config.offsetY);
      ctx.lineTo(sx, sy);
      initialPointSet = true;
    } else {
      ctx.lineTo(sx, sy);
    }
  }
}

```

```

    }
}
if (initialPointSet) {
    ctx.lineTo(rightScreenX, config.offsetY);
    ctx.closePath();
    ctx.fill();
}

// 2. Plot Boundary vertical limits lines a and b
ctx.lineWidth = 1.5;
ctx.setLineDash([4, 4]);
ctx.strokeStyle = '#2b6cb0';

[a, b].forEach(limit => {
    let sx = toScreenX(limit);
    ctx.beginPath();
    ctx.moveTo(sx, 0);
    ctx.lineTo(sx, h);
    ctx.stroke();
});
ctx.setLineDash([]); // Reset styling dash arrays

// 3. Draw the function plot curve trace
ctx.strokeStyle = '#4a6fa5';
ctx.lineWidth = 2.5;
ctx.beginPath();

let drawing = false;
for (let sx = 0; sx < w; sx++) {
    let mx = toMathX(sx);
    let my = f(mx);

```

```

    if (!isNaN(my) && isFinite(my)) {
        let sy = toScreenY(my);
        if (!drawing) {
            ctx.moveTo(sx, sy);
            drawing = true;
        } else {
            ctx.lineTo(sx, sy);
        }
    } else {
        drawing = false;
    }
}
ctx.stroke();
}

```

// Synchronize Input triggers between Text boxes and Interactive Range Sliders

```

function syncInputs(source) {
    if (source === 'sliders') {
        inputA.value = sliderA.value;
        inputB.value = sliderB.value;
    } else {
        sliderA.value = inputA.value;
        sliderB.value = inputB.value;
    }
    update();
}

```

// Event Listeners Configuration

```

fxInput.addEventListener('input', update);
inputA.addEventListener('input', () => syncInputs('inputs'));

```

```
inputB.addEventListener('input', () => syncInputs('inputs'));
sliderA.addEventListener('input', () => syncInputs('sliders'));
sliderB.addEventListener('input', () => syncInputs('sliders'));
```

```
// Drag-to-pan Canvas Mouse Hooks
```

```
canvas.addEventListener('mousedown', (e) => {
  isDragging = true;
  startX = e.clientX - config.offsetX;
  startY = e.clientY - config.offsetY;
});
```

```
window.addEventListener('mousemove', (e) => {
  if (!isDragging) return;
  config.offsetX = e.clientX - startX;
  config.offsetY = e.clientY - startY;
  update();
});
```

```
window.addEventListener('mouseup', () => { isDragging = false; });
```

```
// Mouse wheel Zoom bindings
```

```
canvas.addEventListener('wheel', (e) => {
  e.preventDefault();
  const zoomFactor = e.deltaY < 0 ? 1.1 : 0.9;
```

```
// Pin pointer math point coordinates during dynamic zooming adjustments
```

```
const mouseX = e.clientX - canvas.getBoundingClientRect().left;
const mouseY = e.clientY - canvas.getBoundingClientRect().top;
const mathX = toMathX(mouseX);
const mathY = toMathY(mouseY);
```

```
config.zoomX *= zoomFactor;
config.zoomY *= zoomFactor;

// Restore dynamic positioning relative to localized pointer frame adjustments
config.offsetX = mouseX - mathX * config.zoomX;
config.offsetY = mouseY + mathY * config.zoomY;

update();
}, { passive: false });

// Window layout scaling callbacks
window.addEventListener('resize', resizeCanvas);

// System bootup initializing
window.onload = resizeCanvas;
</script>

</body>
</html>
```